In [9]:
```
!pip install mlxtend -q
```

In [10]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confus
from mlxtend.plotting import plot_decision_regions
```

In [11]:
```
import pandas as pd
import zipfile
import urllib.request
import io

url = "https://archive.ics.uci.edu/static/public/222/bank+marketing.zip"

with urllib.request.urlopen(url) as response:
    zip_data = response.read()

with zipfile.ZipFile(io.BytesIO(zip_data)) as z:
    # Find the bank.zip file inside the downloaded ZIP
    bank_zip = [name for name in z.namelist() if name.endswith("bank.zip"

    # Open the inner bank.zip
    with z.open(bank_zip) as f:
        inner_zip_data = f.read()

    with zipfile.ZipFile(io.BytesIO(inner_zip_data)) as inner_z:
        # Find bank.csv inside the inner ZIP
        bank_file = [name for name in inner_z.namelist()
                     if name.lower().endswith("bank.csv")][0]

        # Load the dataset
        with inner_z.open(bank_file) as f:
            bank = pd.read_csv(f, sep=";")

print("Dataset shape:", bank.shape)
bank.head()
```

```
Dataset shape: (4521, 17)
```

Out[11]:

	age	job	marital	education	default	balance	housing	loan	contact
0	30	unemployed	married	primary	no	1787	no	no	cellular
1	33	services	married	secondary	no	4789	yes	yes	cellular
2	35	management	single	tertiary	no	1350	yes	no	cellular
3	30	management	married	tertiary	no	1476	yes	yes	unknown
4	59	blue-collar	married	secondary	no	0	yes	no	unknown

In [12]:
```
X = bank[["age", "balance"]].to_numpy()
y = bank["y"].map({"no": 0, "yes": 1}).to_numpy()
```

```
print("Features used:", ["age", "balance"])
print("Target classes:", ["no", "yes"])
```

```
Features used: ['age', 'balance']
Target classes: ['no', 'yes']
```

In [13]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [14]:
```
model = SVC(
    kernel="linear",
    C=1.0,
    random_state=42
)

model.fit(X_train, y_train)
```

Out[14]:

▾ SVC ⓘ ?

▸ Parameters

▸ Fitted attributes

In [15]:
```
y_pred = model.predict(X_test)
```

In [16]:
```
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy)


print(
    "\nClassification Report:\n",
    classification_report(
        y_test,
        y_pred,
        target_names=["no", "yes"]
    )
)
```

```
Model Accuracy: 0.8585635359116022

Classification Report:
               precision    recall  f1-score   support

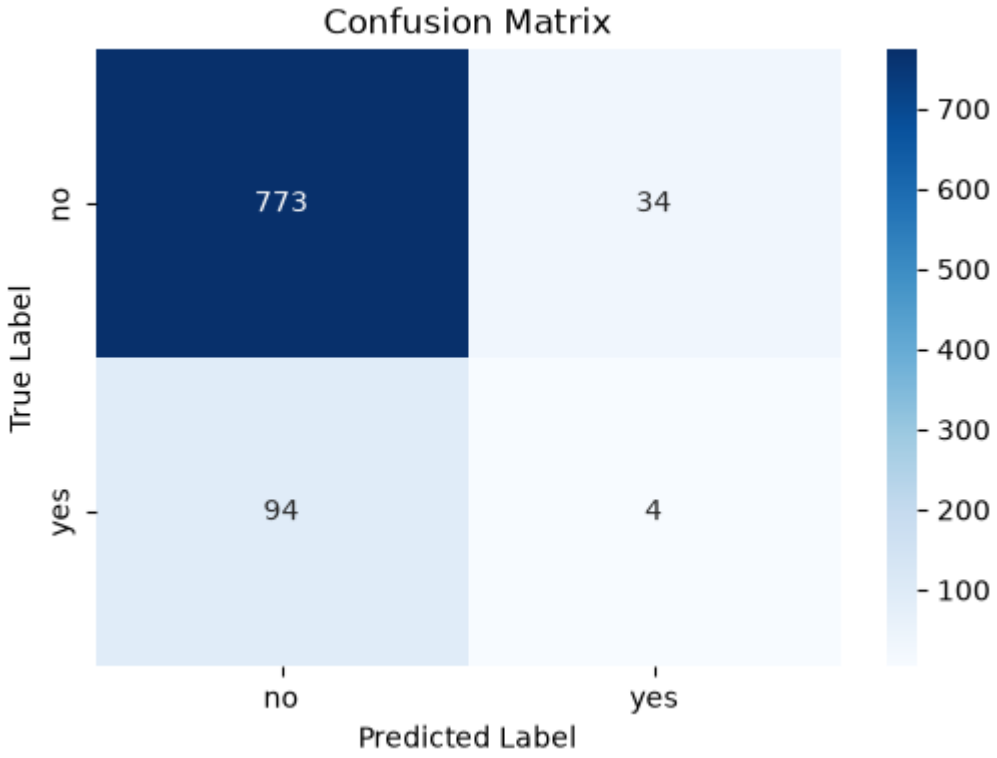
           no       0.89      0.96      0.92       807
          yes       0.11      0.04      0.06        98

    accuracy                           0.86       905
   macro avg       0.50      0.50      0.49       905
weighted avg       0.81      0.86      0.83       905
```

In [17]:
```
conf_matrix = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 4))
sns.heatmap(
    conf_matrix,
    annot=True,
```

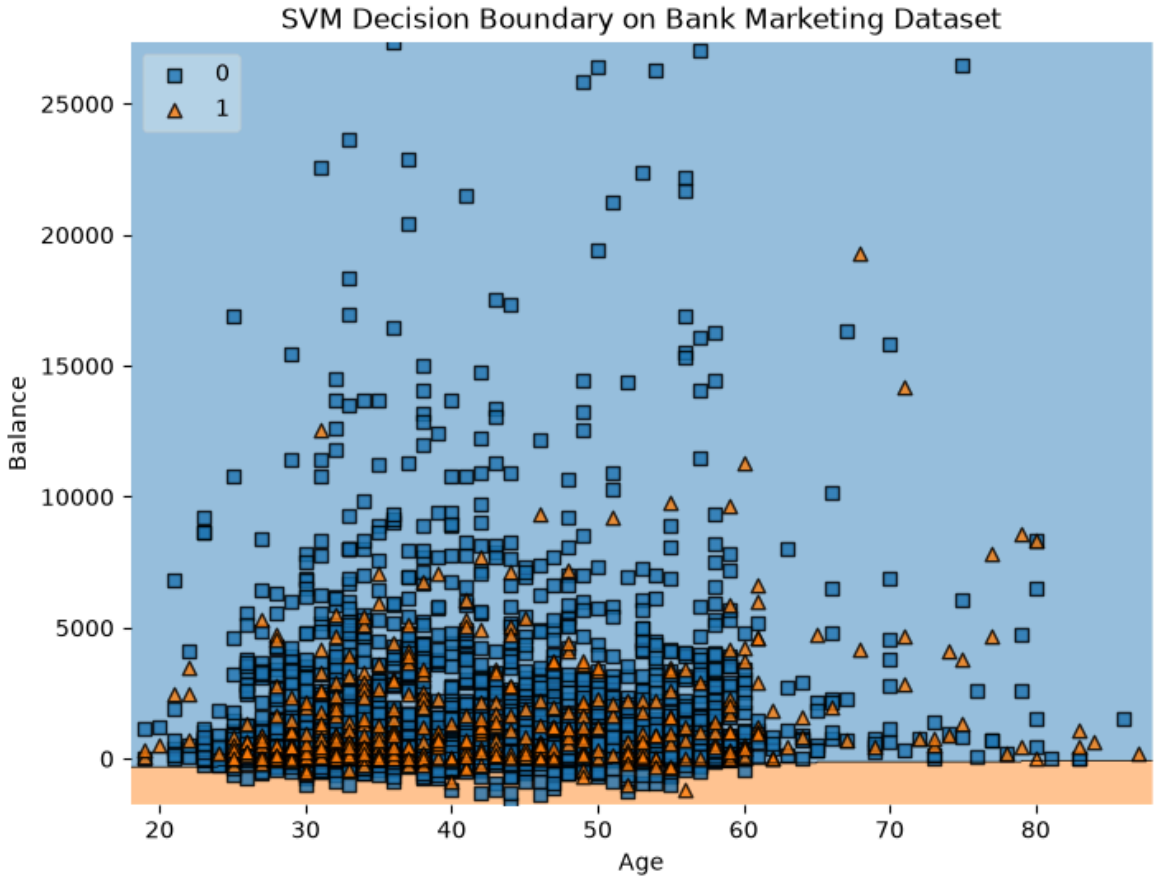
```
    cmap="Blues",
    fmt="d",
    xticklabels=["no", "yes"],
    yticklabels=["no", "yes"]
)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()
```



In [18]:
```
plt.figure(figsize=(8, 6))

plot_decision_regions(
    X_train,
    y_train,
    clf=model,
    legend=2
)

plt.xlabel("Age")
plt.ylabel("Balance")
plt.title("SVM Decision Boundary on Bank Marketing Dataset")
plt.show()
```


SVM Decision Boundary on Bank Marketing Dataset
0
1
25000
20000
15000
10000
5000
0
Balance
20
30
40
50
60
70
80
Age

In [ ]: